

Complete Working of the ML Code

Shop Smart E-Commerce — Revenue Classification using Preprocessing, Decision Tree, Pipeline and GridSearchCV

Classification

Decision Tree

ColumnTransformer

Pipeline

GridSearchCV

Cross Validation

1. The Big Picture

The purpose of this code is to build a machine-learning classification model that predicts **Revenue** using information about an e-commerce visitor.

Data → **X/y Separation** → **Train/Test Split** → **Preprocessing** → **Decision Tree** → **GridSearchCV** → **Best Model** → **Prediction** → **Evaluation**

Conceptually, the system works like this:

INPUT FEATURES



Website visitor information



ML MODEL



Revenue prediction



0 = No revenue / purchase

1 = Revenue / purchase

Contents

1. [Import Libraries](#)
2. [Load Dataset](#)
3. [Inspect Data](#)
4. [Separate X and y](#)
5. [Identify Feature Types](#)
6. [Train-Test Split](#)
7. [Preprocessing](#)
8. [Decision Tree](#)
9. [Pipeline](#)
10. [Parameter Grid](#)
11. [GridSearchCV](#)
12. [Training](#)
13. [Prediction](#)
14. [Evaluation](#)
15. [Complete Mental Model](#)

2. Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
```

```
f1_score,  
classification_report,  
confusion_matrix,  
accuracy_score  
)
```

Pandas

`pandas` is used for working with structured/tabular data.

Examples:

```
pd.read_csv()  
df.head()  
df.info()
```

Matplotlib

Matplotlib is used for creating visualizations and graphs.

Seaborn

Seaborn is another visualization library commonly used for statistical plots.

`train_test_split`

Splits the dataset into training and testing data.

GridSearchCV

Searches through different hyperparameter combinations and uses cross-validation to determine which configuration performs best according to the selected metric.

ColumnTransformer

Allows different preprocessing operations to be applied to different groups of columns.

StandardScaler

Standardizes numerical features using their mean and standard deviation.

OneHotEncoder

Converts categorical values into numerical indicator columns.

Pipeline

Combines multiple ML operations into one workflow.

DecisionTreeClassifier

The machine-learning classification algorithm used in this project.

Evaluation Metrics

F1 score, classification report, confusion matrix and accuracy are used to evaluate the model.

3. Load the Dataset

```
df = pd.read_csv("shop_smart_ecommerce.csv")
```

This reads the CSV file and stores the complete dataset inside the Pandas DataFrame named `df`.

```
shop_smart_ecommerce.csv
    ↓
pandas
    ↓
df
```

4. Inspect the Data

```
df.head()
```

Displays the first five rows of the dataset.

```
df.info()
```

Shows useful information such as:

- Column names
- Number of entries
- Data types
- Non-null values

5. Separate Features X and Target y

```
X = df.drop("Revenue", axis=1)
```

```
y = df["Revenue"].astype(int)
```

This is one of the most important concepts in machine learning.

Variable	Meaning
X	Input features used by the model to make predictions.
y	Target/output that the model is trying to predict.

Suppose the dataset contains:

```
Age  
Income  
Month
```

```
VisitorType
Weekend
Revenue
```

Then:

```
X =
Age
Income
Month
VisitorType
Weekend

y =
Revenue
```

Remember: X = Features / Inputs

y = Target / Output

6. Identify Numerical and Categorical Features

```
num_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

cat_features = X.select_dtypes(
    include=["object", "category"]
).columns
```

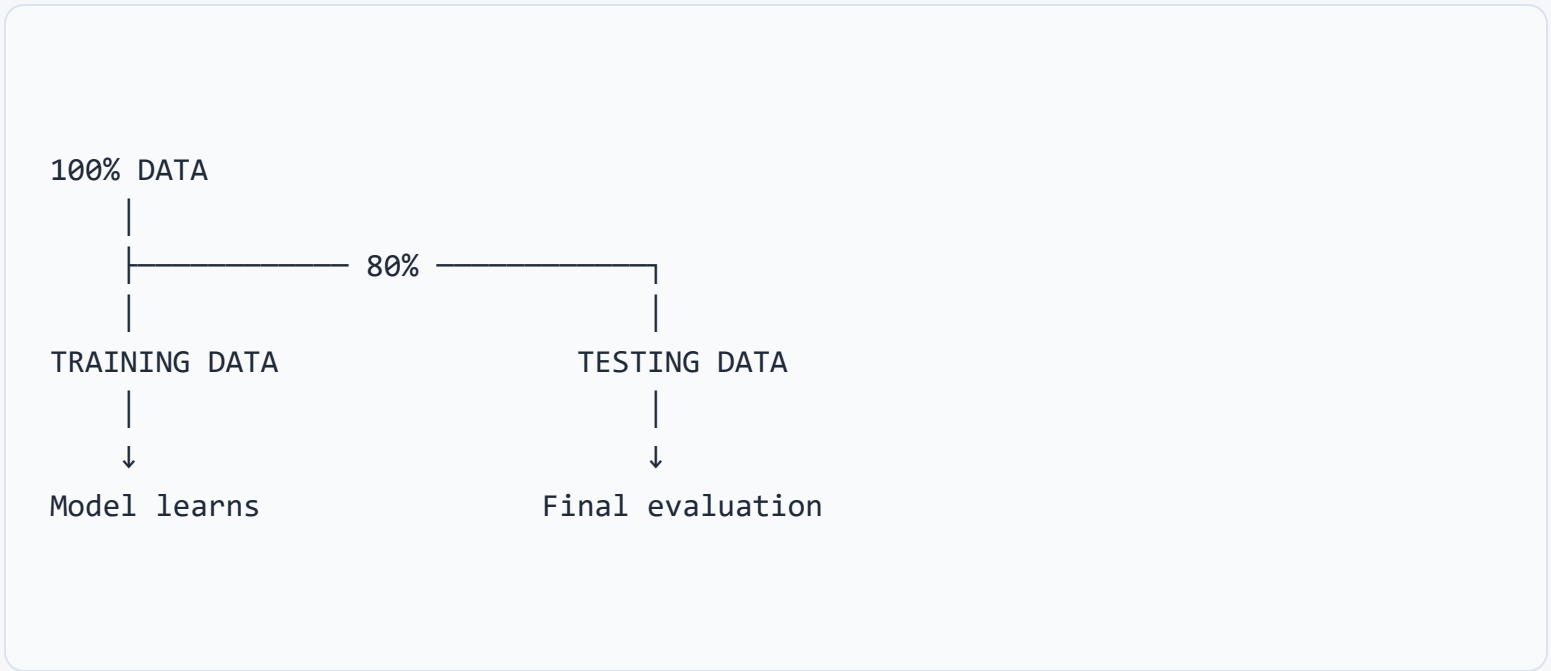
The program automatically separates columns based on their data types.

Feature Type	Examples	Preprocessing
Numerical	Age, Income, PageVisits	StandardScaler

Categorical	Month, VisitorType, Browser	OneHotEncoder
-------------	-----------------------------	---------------

7. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=32,  
    stratify=y  
)
```



test_size=0.2

20% of the data is reserved for testing and approximately 80% is used for training.

random_state=32

Makes the split reproducible.

stratify=y

Attempts to preserve the class distribution of the target variable in both the training and testing sets.

8. Build the Preprocessing Pipeline

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("numerical", StandardScaler(), num_features),  
  
        ("categorical",  
         OneHotEncoder(handle_unknown="ignore"),  
         cat_features)  
    ]  
)
```

The idea is:

Numerical columns

↓

StandardScaler

↓

Scaled numerical values

Categorical columns

↓

OneHotEncoder

↓

Numerical encoded representation

Why ColumnTransformer?

Because numerical and categorical columns require different preprocessing techniques.

Numerical Branch

```
("numerical", StandardScaler(), num_features)
```

This means:

num_features → **StandardScaler** → transformed numerical features

Categorical Branch

```
(  
    "categorical",  
    OneHotEncoder(handle_unknown="ignore"),  
    cat_features  
)
```

This means:

cat_features → **OneHotEncoder** → encoded numerical features

What does handle_unknown="ignore" mean?

Imagine the training data contains:

```
Chrome  
Firefox  
Safari
```

But later the test data contains a category that wasn't seen during training:

```
Edge
```

With:

```
handle_unknown="ignore"
```

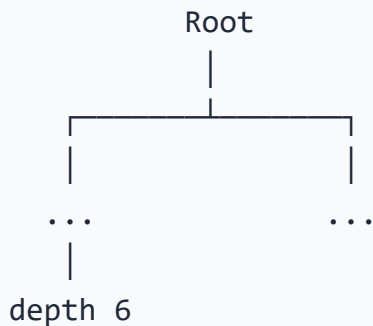
the encoder handles the unseen category safely instead of throwing an error.

9. Create the Decision Tree

```
dt = DecisionTreeClassifier(  
    max_depth=6,  
    min_samples_leaf=30,  
    class_weight="balanced",  
    random_state=42  
)
```

max_depth=6

Limits the maximum depth of the Decision Tree.



Limiting tree depth can reduce unnecessary complexity and help control overfitting.

min_samples_leaf=30

Requires each leaf node to contain at least 30 training samples. This can prevent extremely small leaves and overly specific rules.

class_weight="balanced"

If the target classes are imbalanced, this automatically adjusts class weights so that the minority class receives more importance during training.

random_state=42

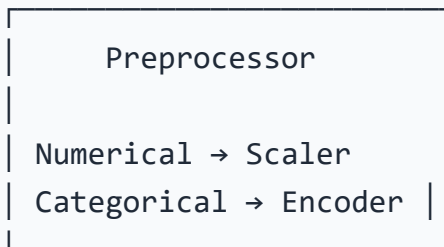
Makes randomized behavior reproducible.

10. Build the Pipeline

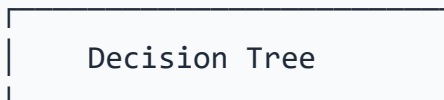
```
pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", dt)
])
```

This combines preprocessing and model training into one object.

RAW DATA



Processed Features



Prediction

The Pipeline is especially useful because the same preprocessing workflow can be consistently applied during training, cross-validation and prediction.

11. Create the Hyperparameter Grid

```
param_grid = {  
    "model__max_depth": [4, 6, 8],  
    "model__min_samples_leaf": [20, 30, 50]  
}
```

You are telling GridSearchCV which values to test.

max_depth	min_samples_leaf
4	20, 30, 50
6	20, 30, 50
8	20, 30, 50

3 max_depth values × 3 min_samples_leaf values = **9 combinations**

Why model__max_depth?

Your Pipeline contains:

```
Pipeline([  
    ("preprocessor", preprocessor),  
    ("model", dt)  
])
```

The model step is named:

model

Therefore, to access a parameter inside the model:

```
step_name__parameter_name
```

So:

```
model__max_depth
```

means: **access max_depth inside the Pipeline's model step.**

12. GridSearchCV

```
grid = GridSearchCV(  
    pipe,  
    param_grid,  
    scoring="f1",  
    cv=5,  
    n_jobs=-1  
)
```

scoring="f1"

GridSearchCV uses F1 score to compare the different hyperparameter configurations.

cv=5

Uses 5-fold cross-validation.

Training Data

```
|  
├─ Fold 1  
├─ Fold 2  
├─ Fold 3  
└─ Fold 4
```

└─ Fold 5

Example:

Train → 2,3,4,5

Validate → 1

Train → 1,3,4,5

Validate → 2

Train → 1,2,4,5

Validate → 3

Train → 1,2,3,5

Validate → 4

Train → 1,2,3,4

Validate → 5

Every hyperparameter configuration is evaluated across all five folds and its scores are averaged.

9 combinations × 5 folds = **45 model fits**

n_jobs=-1

Allows GridSearchCV to use all available CPU cores where supported, which can speed up the search.

13. Train GridSearchCV

```
grid.fit(X_train, y_train)
```

This is where the actual training and hyperparameter search occur.

```
X_train + y_train
  ↓
GridSearchCV
  ↓
9 hyperparameter combinations
  ↓
5-fold CV for each
  ↓
45 fits
  ↓
Compare average F1
  ↓
Select best configuration
```

What happens during each fit?

```
Training Fold
  ↓
ColumnTransformer
  ├── Numerical → StandardScaler
  └── Categorical → OneHotEncoder
  ↓
Decision Tree
  ↓
Validation Prediction
  ↓
F1 Score
```

Because preprocessing is inside the Pipeline, it is fitted appropriately within each cross-validation training fold. This helps prevent validation-data leakage during model selection.

14. Get the Best F1 Score

```
print("Best F1:", grid.best_score_)
```

`grid.best_score_` contains the best mean cross-validation score found by GridSearchCV.

Example:

```
Best F1: 0.742
```

This means the selected configuration had a mean CV F1 score of approximately 0.742 across the five validation folds.

15. Get the Best Hyperparameters

```
print("Best params:", grid.best_params_)
```

Example output:

```
{
  'model__max_depth': 6,
  'model__min_samples_leaf': 30
}
```

This means that, among the values supplied to GridSearchCV, this combination produced the highest mean cross-validation F1 score.

16. Predict on the Test Set

```
y_pred = grid.predict(X_test)
```

The test set is passed through the best fitted Pipeline.

```
X_test
↓
Best fitted Pipeline
↓
Preprocessing
↓
Best Decision Tree
↓
Predicted Revenue
↓
y_pred
```

The test set represents data that was not used to fit the model during training.

17. Evaluate the Model

F1 Score

```
f1_score(y_test, y_pred)
```

Compares actual labels with predicted labels using the F1 metric. F1 combines precision and recall into one metric.

Classification Report

```
classification_report(y_test, y_pred)
```

Provides precision, recall, F1 score and support for each class.

Example:

	precision	recall	f1-score
0	0.95	0.96	0.95
1	0.72	0.68	0.70
accuracy	0.91		

This lets you inspect the performance separately for Revenue = 0 and Revenue = 1.

Confusion Matrix

```
confusion_matrix(y_test, y_pred)
```

For binary classification:

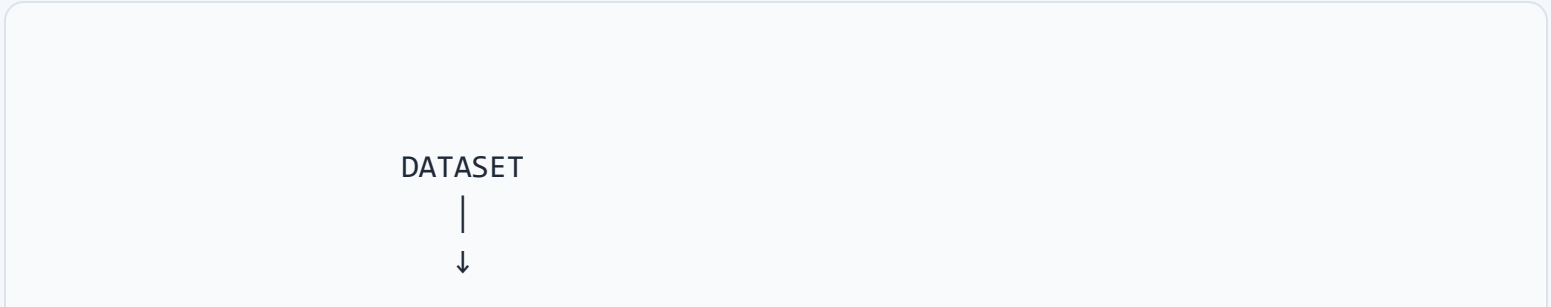
	Predicted 0	Predicted 1
Actual 0	True Negative (TN)	False Positive (FP)
Actual 1	False Negative (FN)	True Positive (TP)

Accuracy

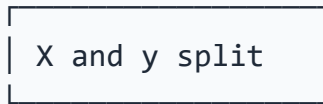
```
accuracy_score(y_test, y_pred)
```

Accuracy measures the proportion of all predictions that are correct.

18. The Entire Workflow



shop_smart_ecommerce.csv



X = features

y = Revenue

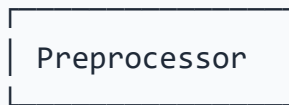


Train/Test Split



X_train
y_train

X_test
y_test



Numerical

Categorical



StandardScaler

OneHotEncoder



Decision Tree



GridSearchCV



9 combinations

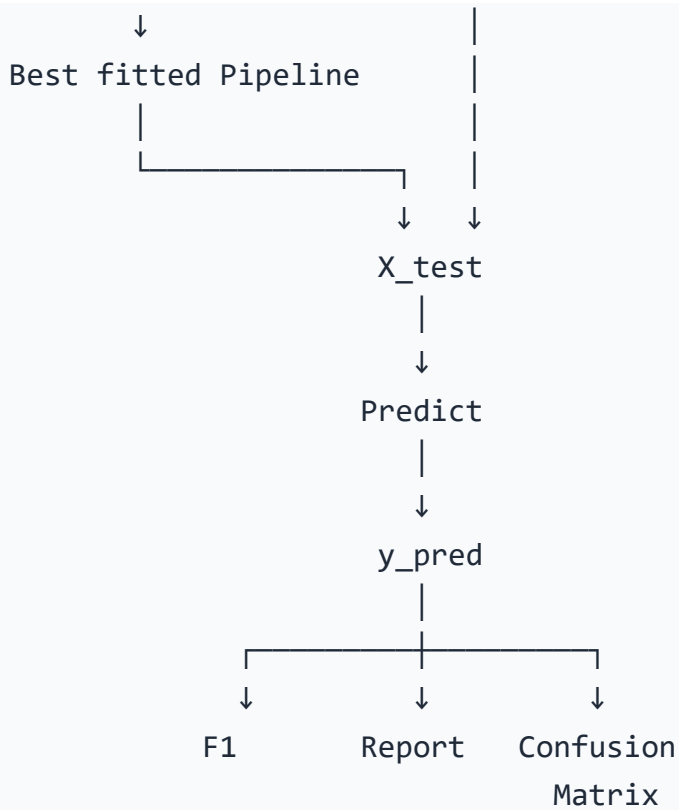
×

5-fold CV



Best Hyperparameters





19. Parameters vs Hyperparameters

This distinction is very important in machine learning.

Hyperparameters

Values that control the configuration or learning behavior of the model and are selected before/during training.

```
max_depth=6
min_samples_leaf=30
```

GridSearchCV searches through the values supplied for these hyperparameters.

Model Parameters

Values learned by the model from the training data.

For a Decision Tree, the model learns decision rules and split thresholds from the training data.

Simple rule:

Hyperparameters control/configure how the model learns.

Parameters are learned by the model from the data.

20. Why Use Pipeline + GridSearchCV?

Pipeline answers:

What sequence of operations should happen?

```
Raw Data
  ↓
Preprocessing
  ↓
Model
```

GridSearchCV answers:

Which hyperparameter configuration should I use?

```
Configuration 1
Configuration 2
Configuration 3
...
Configuration 9
  ↓
Compare CV F1
  ↓
Choose best configuration
```

Pipeline = organizes the ML workflow.

GridSearchCV = searches for a good hyperparameter configuration.

21. Why Don't We Manually Preprocess X_test?

Because the Pipeline handles the transformation automatically.

```
grid.predict(X_test)
```

Conceptually:

```
X_test
↓
Apply fitted preprocessing transformations
↓
Transform into the same feature representation
↓
Use the best fitted Decision Tree
↓
Generate prediction
```

This means you don't need to manually call the scaler and encoder every time you make a prediction.

22. Important Correction in the Original Code

Main problem in the original version:

You created a DecisionTreeClassifier:

```
dt = DecisionTreeClassifier(...)
```

but then your Pipeline contained:

```
("model", LogisticRegression())
```

while your GridSearch tried to tune:

```
model__max_depth  
model__min_samples_leaf
```

Those parameters belong to DecisionTreeClassifier, not LogisticRegression.

Correct Decision Tree Pipeline

```
pipe = Pipeline([  
    ("preprocessor", preprocessor),  
    ("model", dt)  
])
```

Now these parameters correctly refer to the Decision Tree:

```
param_grid = {  
    "model__max_depth": [4, 6, 8],  
    "model__min_samples_leaf": [20, 30, 50]  
}
```

23. The 10 Things You Should Remember

1. **Load the data.**
2. **Separate X and y.**
3. **Identify numerical and categorical features.**
4. **Split the data into training and testing sets.**
5. **Build the preprocessing workflow.**

6. **Create the machine-learning model.**
7. **Combine preprocessing and model using Pipeline.**
8. **Use GridSearchCV to tune hyperparameters.**
9. **Predict on unseen test data.**
10. **Evaluate using F1, accuracy, classification report and confusion matrix.**

24. Final Mental Model

Training Data → Learn

Cross-Validation → Compare / Tune Configurations

Test Data → Final Evaluation on Unseen Data

The central idea is:

The model learns from training data.

GridSearchCV helps select hyperparameters using cross-validation.

The test set remains separate and is used at the end to estimate how the final selected model performs on unseen data.

Quick Revision

```
DATA
↓
X / y
↓
Train-Test Split
↓
Identify Numerical + Categorical Columns
```

```
↓
ColumnTransformer
├─ Numerical → StandardScaler
└─ Categorical → OneHotEncoder

↓
Pipeline
├─ Preprocessor
└─ Decision Tree

↓
GridSearchCV
├─ Hyperparameter combinations
├─ 5-fold Cross Validation
└─ F1 scoring

↓
Best Model
↓
X_test
↓
Prediction
↓
Evaluation
├─ F1 Score
├─ Accuracy
├─ Classification Report
└─ Confusion Matrix
```